CORE JAVA SERIES — DAY 16

The Collection and List Interfaces

What every container can do through the Collection interface, the difference between Collection, Collections and Iterable, then the List interface in full — index methods, ListIterator, immutable lists — and the four List classes: ArrayList, LinkedList, Vector and Stack.

The Collection Interface

`Collection` extends `Iterable` and is the parent of every data-structure interface: `List`, `Set` and `Queue`. Its purpose is to declare what *any* container can do, whatever it uses inside. That is inheritance and polymorphism at work: one reference type can hold any concrete class.

```
Collection<Integer> c = new ArrayList<>();   // or LinkedList, HashSet, ArrayDeque ...
c.add(1); c.add(2); c.add(3);
c.size();                                    // every method of Collection is available
```

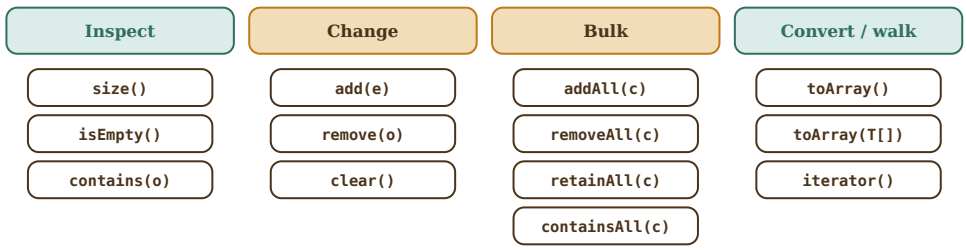
Code that only needs "a bunch of things" can therefore be written against `Collection` and will work with any data structure passed in.

Three similar names

Name	What it is
`Collection`	The **interface**: the common operations of all containers.
`Collections`	A **utility class** of static helper methods (sorting, searching and so on), studied later.
`Iterable`	The parent interface above `Collection`; it only promises traversal.

The same pattern appears elsewhere in Java: `Object` is the parent of all classes and `Objects` is a utility class. A trailing "s" usually means a utility class.

You do not need to memorise the method list. Think of the basic things you could ask any container to do, and group them:



Anything every container can do lives in Collection; list-, set- and queue-only methods live below it.

The Collection methods fall into four natural groups.

Inspecting: size, isEmpty, contains

```
Collection<Integer> c = new ArrayList<>();
c.add(1); c.add(2); c.add(3);

System.out.println(c.size());        // 3
System.out.println(c.isEmpty());     // false
System.out.println(c.contains(2));   // true
```

`isEmpty()` is preferred over `size() == 0` because each structure can answer it in the cheapest way it knows.

`contains(Object o)` takes an `Object`, not `E`. Membership is decided by `equals()`, and `equals()` lives in `Object`, so any object can be passed. How fast it is depends on the structure: a list walks its elements comparing each one (**O(n)**), while a hash-based set hashes the argument and looks in one bucket (**O(1)**).

Converting to an array: toArray

`toArray()` copies any collection into an array, which gives continuous memory and random access. The no-argument version returns `Object[]`, which is awkward because you can only call `Object` methods on the elements without casting. The overload `toArray(T[] a)` returns an array of the type you want:

```
Object[] objs = c.toArray();                    // elements typed as Object
for (Object o : objs) System.out.println(o);    // 1 2 3

Integer[] nums = c.toArray(new Integer[0]);     // elements typed as Integer
for (Integer i : nums) System.out.println(i);   // 1 2 3
```

> `new Integer[0]` is only a **type marker**. A zero-length array occupies no useful memory; it just tells `toArray` which array type to build and return.

Changing: add, remove, clear

`add(E e)` returns a `boolean`: **true** if the collection changed, **false** if the element was not added. A list always accepts the element; a `HashSet` refuses a duplicate:

```
Collection<Integer> s = new HashSet<>(List.of(1, 2, 3));
System.out.println(s.add(4));    // true   (new element)
System.out.println(s.add(3));    // false  (duplicate, set unchanged)
```

`remove(Object o)` also takes an `Object` (again, comparison uses `equals()`) and returns whether something was removed. If duplicates exist, it removes only the **first occurrence**.

```
System.out.println(s.remove(2));    // true
System.out.println(s.remove(9));    // false  (not present)

Collection<Integer> d = new ArrayList<>(List.of(1, 2, 3, 3));
d.remove(3);
System.out.println(d);              // [1, 2, 3]   only the first 3 went
```

`clear()` empties the container completely.

Bulk operations

Method	Meaning	Set-theory view
`addAll(Collection<? extends E> c)`	Add every element of `c`; true if anything was added.	union
`containsAll(Collection<?> c)`	Are all elements of `c` present?	subset test
`removeAll(Collection<?> c)`	Remove every element that appears in `c`.	difference
`retainAll(Collection<?> c)`	Keep *only* the elements that appear in `c`.	intersection

`? extends E` means "`E` or any subtype": if the collection holds `Animal` you may add a collection of `Dog`. The checking methods accept `Collection<?>` because they only compare with `equals()`, so the element type does not matter.

```
Collection<Integer> c = new ArrayList<>(List.of(1, 2, 3));
c.addAll(List.of(5, 6, 7, 8, 9));
System.out.println(c);                              // [1, 2, 3, 5, 6, 7, 8, 9]
System.out.println(c.containsAll(List.of(1, 2, 3)));// true

c.removeAll(List.of(1, 2));
System.out.println(c);                              // [3, 5, 6, 7, 8, 9]

c.retainAll(List.of(3, 5));
System.out.println(c);                              // [3, 5]

c.clear();
System.out.println(c);                              // []
```

> Printing a collection directly shows `[1, 2, 3]` because every collection overrides `toString()`. No loop is needed just to look at the contents.

Methods inherited from Object

`Collection` equality compares *contents*, so the collection classes override `equals()` (the default `Object` version compares references). The rule you must honour in your own classes: **if two objects are equal by `equals()`, their `hashCode()` values must be equal** (the reverse need not hold). If you override `equals()` in a class such as `Student`, override `hashCode()` too.

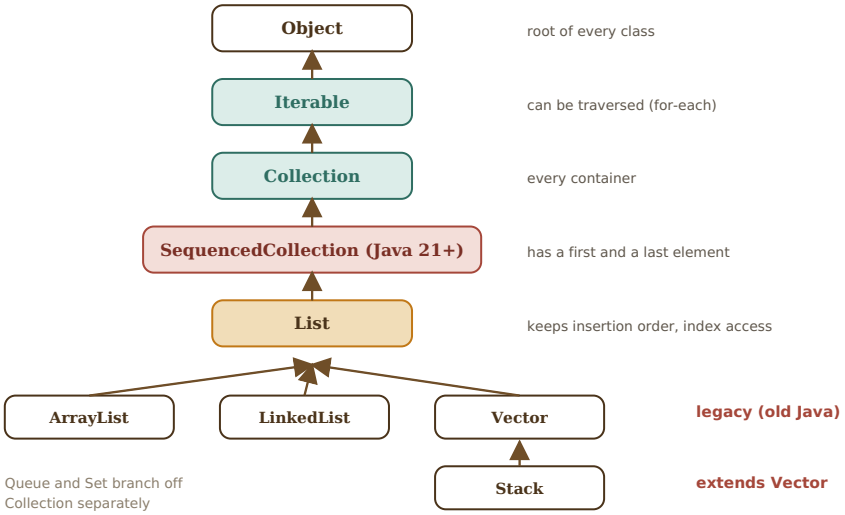
Default methods added in modern Java

`removeIf`, `stream`, `parallelStream` and `spliterator` are *default* methods. Modern Java adds new interface methods as defaults so older code that implements the interface does not break. `removeIf` takes a lambda (functional interfaces, later); the stream methods belong to the Streams topic.

Most of what you can do with a container is declared right here. The sub-interfaces add only what is specific to them: a queue adds front/back operations, a sorted set adds ordering operations, and a list adds position-based operations — which is the next topic.

The List Interface

A list is a collection with two defining properties: it **preserves insertion order** and it lets you **access elements by position** (index). Under `List` there are four concrete classes:



The List family. Since Java 21 SequencedCollection sits between Collection and List.

`List` is an interface, so it only declares methods; the concrete classes override them. That is why you can write `List<Integer> l = new ArrayList<>()` or `new LinkedList<>()` and use the same calls.

> A `LinkedList` has no real indexes, yet it is still a `List`. It *mimics* indexing: to reach position 2 it starts at the head and hops until it arrives. The result is correct but costs O(n), whereas an `ArrayList` jumps there directly in O(1).

Methods that come from SequencedCollection (Java 21+)

Because a list is a sequence, it has a defined first and last element. These six methods operate on the ends:

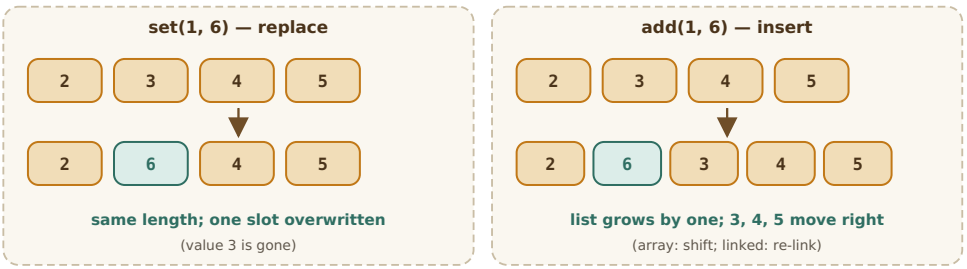
Method	Does
`getFirst()` / `getLast()`	Return the first / last element.
`addFirst(e)` / `addLast(e)`	Insert at the front / back.
`removeFirst()` / `removeLast()`	Remove and return the first / last element.

Position-based methods of List

Method	Does
`E get(int index)`	Return the element at the index.
`E set(int index, E e)`	**Replace** the element at the index.
`void add(int index, E e)`	**Insert** at the index; later elements move one place right.
`boolean addAll(int index, Collection c)`	Insert a whole collection starting at the index.
`E remove(int index)`	Remove by **position** (the Collection version removes by *value*).
`int indexOf(Object o)` / `lastIndexOf(Object o)`	First / last index of a value, found with `equals()`; -1 if absent.
`ListIterator<E> listIterator()` / `listIterator(int index)`	A list-specific iterator (below).
`List.of(...)` / `List.copyOf(c)`	Create an **unmodifiable** list.

set versus add at an index

Take the list `[2, 3, 4, 5]`. `set(1, 6)` overwrites position 1, so the length stays 4. `add(1, 6)` inserts, so the length becomes 5 and everything from position 1 shifts right. An `ArrayList` does that by shifting array slots; a `LinkedList` walks to the position, creates a node and re-links the pointers, with no shifting.



set replaces a slot; add inserts and pushes the rest along.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
System.out.println(list.get(1));          // 2

list.set(1, 5);
System.out.println(list);                 // [1, 5, 3]

list.addAll(0, List.of(9, 8, 7));
System.out.println(list);                 // [9, 8, 7, 1, 5, 3]

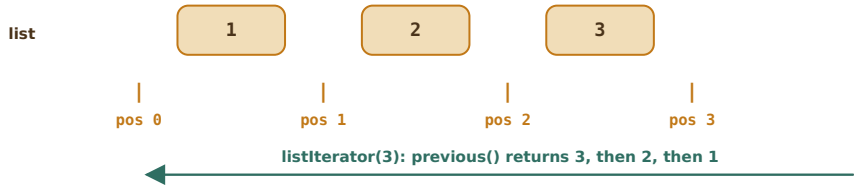
list.remove(0);                           // by index
System.out.println(list);                 // [8, 7, 1, 5, 3]
System.out.println(list.indexOf(5));      // 3
System.out.println(list.lastIndexOf(5));  // 3
```

remove overload trap. In a `List<Integer>`, `remove(1)` means *remove the element at index 1*, because an `int` argument selects `remove(int index)`. To remove the *value* 1, pass an object: `remove(Integer.valueOf(1))`. For `[10, 20, 30, 1]`, `remove(1)` leaves `[10, 30, 1]`.

Cost of the two removes: `remove(index)` is O(1) to find in an `ArrayList` (but elements still shift) and O(n) to reach in a `LinkedList`. `remove(Object)` must search by value, so it is O(n) in both.

ListIterator

The plain `Iterator` can only move forward and remove. A `ListIterator` can also go backward (`hasPrevious()`, `previous()`), report positions (`nextIndex()`, `previousIndex()`), and **add** or **set** elements while walking — all without a `ConcurrentModificationException`. `listIterator(int index)` starts the cursor at a chosen position. The cursor sits *between* elements; `previous()` returns the element just before it.



next() / previous() move the cursor, add() inserts at the cursor,
set() / remove() act on the item just returned

Starting a ListIterator at the end lets it walk the list backwards.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));

ListIterator<Integer> it = list.listIterator(3);     // cursor after the last element
while (it.hasPrevious()) {
    System.out.print(it.previous() + " ");            // 3 2 1
}

ListIterator<Integer> li = list.listIterator();
while (li.hasNext()) {
    if (li.next() == 2) li.add(99);                   // insert while walking: safe
}
System.out.println(list);                             // [1, 2, 99, 3]
```

Unmodifiable lists: List.of and List.copyOf

`List.of(...)` builds a list from the given elements; `List.copyOf(collection)` copies an existing collection. Both return **unmodifiable** lists: any attempt to change them throws `UnsupportedOperationException`.

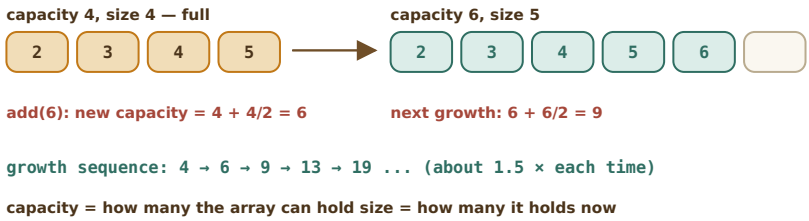
```
List<Integer> fixed = List.of(1, 2, 3, 4, 5, 6, 7, 8);
System.out.println(fixed);          // [1, 2, 3, 4, 5, 6, 7, 8]
fixed.add(9);                       // UnsupportedOperationException

List<Integer> copy = List.copyOf(list);
copy.add(4);                        // UnsupportedOperationException
```

ArrayList

`ArrayList` is a dynamic array. When it is full it allocates a bigger array, copies the old values across and discards the old array. We built a version that doubles the size; Java's real `ArrayList` grows by about **1.5×**:

```
newCapacity = oldCapacity + (oldCapacity / 2)      // 4 -> 6 -> 9 -> 13 -> 19 ...
```



Growth is gradual, and capacity is not the same as size.

Capacity is how many elements the internal array can hold; **size** is how many it currently holds. A fresh array of capacity 4 has size 0; after four adds both are 4; the fifth add triggers growth to capacity 6 while size becomes 5.

Costs

Operation	Cost	Why
`get(i)`, `set(i, e)`	O(1)	random access: jump straight to the slot
`add(i, e)`, `remove(i)`	O(n)	elements must shift right / left
`contains`, `indexOf`, `remove(Object)`	O(n)	compare element by element with equals()

About 90% of the lists used in real code are `ArrayList`s: random access, a cache-friendly contiguous layout and a simple structure.

Constructors

```
List<Integer> a = new ArrayList<>();                // empty list
List<Integer> b = new ArrayList<>(10);              // with an initial capacity
List<Integer> c = new ArrayList<>(List.of(1, 2, 3)); // copy of another collection
```

Methods that belong to ArrayList itself

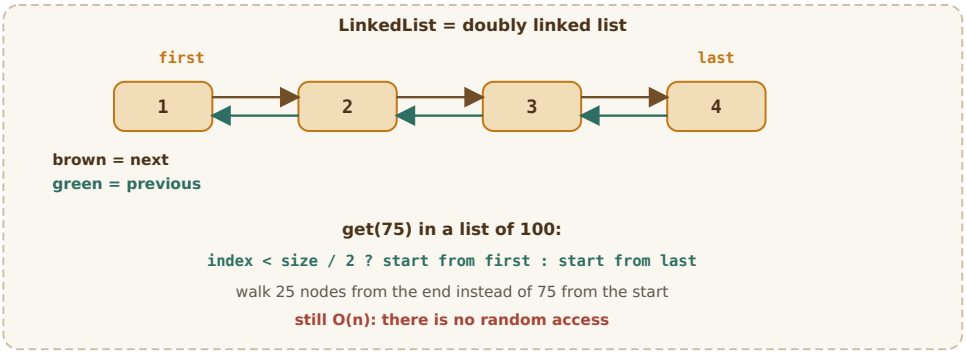
Everything else comes from the interfaces. Two capacity tools are specific to `ArrayList`:

Method	Purpose
`ensureCapacity(n)`	Grow once, up front, to at least `n`. If you know you will add about 100 elements, this avoids many repeated grow-and-copy steps.
`trimToSize()`	Shrink the internal array so capacity equals size, releasing unused slots. Useful when a big list will not grow any more.

LinkedList

A linked list stores each value in a `Node` object (a static nested class) allocated on the heap, joined by references. A singly linked node only knows the next node. Java's `LinkedList` is **doubly linked**: each node holds both `next` and `previous`, so it can be traversed in both directions — which is exactly what a backward `ListIterator` needs.

It keeps references to the **first** and **last** nodes. These are never moved while traversing; the code walks with a temporary reference instead, so the ends are not lost. For `get(index)` it checks which end is closer: if `index < size / 2` it starts from the first node, otherwise from the last. In a list of 100, reaching index 75 costs 25 hops from the end instead of 75 from the start.



Two-way links and two end references let a LinkedList start from the nearer end.

Costs: reaching any position is **O(n)**, so `get`, `set`, `add(i, e)` and `remove(i)` are all O(n). The one real benefit is that inserting or removing needs **no shifting**, only pointer changes once you are there. (Adding or removing at the very ends is the cheap case.)

Because it is easy to work from both ends, `LinkedList` also implements the queue interfaces and carries stack/queue-style methods; those appear when queues and stacks are studied.

> In production code `LinkedList` is rare (under about 10% of list usage). Reads far outnumber writes, and `get` is exactly where a linked list is slow; nearly everything it can do, an `ArrayList` does as well or better.

Vector and Stack: legacy classes

`Vector` was the dynamic array of old Java, from before the Collections Framework existed, and `Stack` is a class that **extends** `Vector` (it does not implement `List` directly). When the framework arrived, `Vector` was retrofitted to implement `List`, which brought both into the hierarchy. A third old class, `Hashtable`, has been replaced by `HashMap`.

They are rarely used today because their methods are dated and because they are **thread-safe** (usable from multiple threads), which adds locking overhead and makes them slower. Where thread safety is needed, modern thread-safe list alternatives exist (covered with concurrency).

Need	Use today	Instead of
Dynamic array / list	`ArrayList`	`Vector`
Stack (LIFO)	`ArrayDeque` (it implements both queue and stack behaviour)	`Stack`
Key-value table	`HashMap`	`Hashtable`

ArrayList versus LinkedList

	ArrayList	LinkedList
Internal structure	dynamic array	doubly linked nodes
get(i) / set(i, e)	O(1)	O(n)
add / remove at an index	O(n) — shifting	O(n) to reach, then O(1) re-link
Memory layout	contiguous, cache-friendly	scattered nodes
Typical use	default choice (~90%)	rare

Active Recall

Q1. Why can a Collection reference hold an ArrayList, a HashSet or an ArrayDeque?
A. All of them implement Collection, so polymorphism lets one reference type hold any of them, and every Collection method is available on it.

Q2. Collection, Collections, Iterable: what is each?
A. Collection is the interface of common container operations; Collections is a utility class of static helpers; Iterable is the parent interface that only promises traversal.

Q3. Why do contains and remove take Object, and what do they use to compare?
A. Comparison is done with equals(), which is defined in Object, so any object can be passed. remove removes only the first matching element.

Q4. What does add return, and when is it false?
A. A boolean saying whether the collection changed. It is false when nothing was added, e.g. a duplicate in a HashSet.

Q5. Difference between removeAll and retainAll?
A. removeAll deletes the elements that appear in the argument; retainAll keeps only those and deletes everything else (intersection).

Q6. What do toArray() and toArray(new Integer[0]) return?
A. Object[] and Integer[] respectively; the zero-length array is only a type marker.

Q7. How do set(1, 6) and add(1, 6) differ on [2, 3, 4, 5]?
A. set overwrites: [2, 6, 4, 5]. add inserts: [2, 6, 3, 4, 5].

Q8. In a List, what is the difference between remove(1) and remove(Integer.valueOf(1))?
A. remove(1) removes the element at index 1; remove(Integer.valueOf(1)) removes the first element equal to 1.

Q9. What can a ListIterator do that an Iterator cannot?
A. Move backwards, report indexes, start at a chosen position, and add or set elements while iterating.

Q10. What does an unmodifiable list do when you call add?
A. List.of and List.copyOf lists throw UnsupportedOperationException on any modification.

Q11. How does ArrayList grow, and what are ensureCapacity and trimToSize?
A. New capacity = old + old/2 (about 1.5×: 4, 6, 9, 13...). ensureCapacity pre-grows to a minimum; trimToSize shrinks capacity to the size.

Q12. Why is LinkedList doubly linked, and how does get(i) use that?
A. Two-way links allow backward traversal, and with first and last references get(i) starts from whichever end is nearer (index < size / 2). It is still O(n).

Q13. Why are Vector and Stack avoided, and what replaces them?
A. They are legacy, dated and synchronized (slower). Use ArrayList instead of Vector and ArrayDeque instead of Stack.